

Mustache Template Alternatives

1 Nunjucks- modern HTML Templating engines

Nunjucks is a powerful templating engine for JavaScript that was created by Mozilla and is heavily inspired *by Python's Jinja2 framework*. It features block inheritance, autoescaping, macros, and asynchronous control, making it significantly more feature-rich than Mustache while maintaining readable syntax. The engine is available in both **Node.js and modern web browsers**, with a small 8K gzipped runtime when using precompiled templates. Nunjucks supports powerful features like template inheritance where child templates can override blocks defined in parent templates, filters that can be applied to variables using pipe operators, and control structures including if statements and for loops. The templating syntax uses `{% %}` for tags and `{{ }}` for variable output, making it familiar to developers coming from other templating languages.

example: taken from cloud defence app.
`use('/blind/nunjucks', function(req, res) { if(req.url) {
var url_parts = url.parse(req.url, true);`

```
var inj = url_parts.query.inj; var tpl = ""; if('tpl' in url_parts.query) {  
tpl = url_parts.query.tpl; } else {  
tpl = url_parts.query.tpl.replace('elsetpl = inj; nunjucks.render(' +  
tpl + ');', 'elsetpl = inj; nunjucks.render(' +  
tpl + ');'); }  
res.send(tpl); } })
```

References

- Mozilla. (n.d.). *Nunjucks*. <https://mozilla.github.io/nunjucks/>
- Mozilla. (n.d.). *GitHub - mozilla/nunjucks: A powerful templating engine with inheritance, asynchronous control, and more (jinja2 inspired)*. <https://github.com/mozilla/nunjucks>
- <https://www.clouddefense.ai/code/javascript/example/nunjucks>

2 Handlebars

Handlebars is a direct evolution of Mustache that compiles templates into JavaScript functions for faster execution than most other template engines. It **adds helpers, block expressions, and partials to the basic Mustache syntax while maintaining a logic-less philosophy**. Although Handlebars is described as logic-less, it can execute simple logic using built-in helpers such as **if, each, unless, and with**. Block helpers allow developers to define custom helpers that invoke **sections of templates with different contexts, providing extensibility while keeping templates declarative**. The syntax remains similar to Mustache with curly braces, making migration straightforward. Handlebars precompiles templates and includes them as JavaScript code rather

than the handlebars template itself, allowing for faster startup time.

```
<script id="template" type="text/x-handlebars-template">
  <h1>{{title}}</h1>
  <ol>
    {{#each list}}
      <li>{{this.name}}</li>
    {{/each}}
  </ol>
  <p>{{footer}}</p>

  <button onclick="fill_template()">fill template</button>
</script>
```

Figure 1: <https://stackoverflow.com/questions/76766197/how-can-i-create-a-handlebars-template-in-vscode>

References

- Handlebars Team. (n.d.). *Handlebars*. <https://handlebarsjs.com/>
- Handlebars Team. (n.d.). *Introduction | Handlebars*. <https://handlebarsjs.com/guide/>
- Handlebars Team. (n.d.). *GitHub - handlebars-lang/handlebars.js: Minimal templating on steroids*. <https://github.com/handlebars-lang/handlebars.js>

3 EJS (Embedded JavaScript Templates)

EJS is a simple templating language that lets you generate HTML markup (markup referring to the static structure and content) with plain JavaScript (via ejs tags to make it dynamic), without reinventing iteration and control flow. Rather than learning arcane new syntax, developers can simply write JavaScript that emits the HTML they want. EJS uses special delimiters where scriptlet tags (<% %>) insert JavaScript code, output tags (<%= %>) display HTML-escaped values, and unescaped output tags (<%- %>) display raw HTML. EJS caches intermediate JavaScript functions for fast execution and provides plain JavaScript exceptions with template line numbers for easy debugging. The engine includes features for template includes, custom delimiters, and a full-featured command-line interface. EJS is effectively a JavaScript runtime whose entire job is to execute JavaScript, making it powerful but requiring careful input validation when used in production.

EJS Code Example with Highlighted Markup

Below is an EJS template showing the distinction between markup and template tags:

Listing 1: profile.ejs Template

```
1  <!DOCTYPE html>                                /* MARKUP */
2  <html>                                           /* MARKUP */
3  <head>                                           /* MARKUP */
4      <title><%= pageTitle %></title>             /* MARKUP + EJS TAG */
5  </head>                                         /* MARKUP */
6  <body>                                           /* MARKUP */
```

```

7      <h1>Welcome , <%= user.name %>!/h1> /* MARKUP + EJS TAG */
8
9      <p><strong>Email:</strong> <%= user.email %></p> /* MARKUP
      + EJS TAG */
10
11     <!-- Conditional rendering -->
12     <% if (user.role === 'admin') { %> /* EJS TAG (logic) */
13         <div class="admin-panel"> /* MARKUP */
14             <h2>Admin Panel</h2> /* MARKUP */
15             <p>You have admin privileges</p> /* MARKUP */
16         </div> /* MARKUP */
17     <% } else { %> /* EJS TAG (logic) */
18         <p>Regular user</p> /* MARKUP */
19     <% } %> /* EJS TAG (logic) */
20
21     <!-- Loop through hobbies -->
22     <h2>Hobbies:</h2> /* MARKUP */
23     <ul> /* MARKUP */
24         <% user.hobbies.forEach(function(hobby) { %> /* EJS TAG
      (loop) */
25             <li><%= hobby %></li> /* MARKUP + EJS TAG */
26         <% }>; %> /* EJS TAG (loop end) */
27     </ul> /* MARKUP */
28 </body> /* MARKUP */
29 </html> /* MARKUP */

```

Visual Legend

Markup (HTML)

HTML tags that define structure: <html>, <body>, <h1>, <p>, <div>, ,

EJS Template Tags

- <%= variable %> – Output escaped content
- <% code %> – Execute JavaScript logic (if, loops, etc.)
- <%- html %> – Output unescaped HTML

References

- EJS Team. (n.d.). *EJS – Embedded JavaScript templates*. <https://ejs.co/>
- EJS Team. (n.d.). *GitHub - mde/ejs: Embedded JavaScript templates*. <http://ejs.co>
- GeeksforGeeks. (2025). *How to Use Embedded JavaScript (EJS) as a Template Engine in Express JS?* <https://www.geeksforgeeks.org/node-js/how-to-use-embedded-javascript-in-express-js/>
- the code and visual legend is AI generated for understanding and learning purposes <https://claude.ai/chat/9edf7207-2ae7-42dd-97df-288f405c6389>

4 Jinja2

Jinja2 is a fast, expressive, and extensible templating engine for Python with special placeholders that allow writing code similar to Python syntax. **Templates support inheritance and inclusion, macros within templates, HTML autoescaping to prevent XSS attacks, and a sandboxed environment to safely render untrusted templates.** Templates are compiled to *optimized Python code just-in-time and cached (once compiled once, it does not need to be compiled again)*. Created by Armin Ronacher, the same developer behind Flask, Jinja2 draws inspiration from Django's templating system but offers more FLEXIBILITY AND POWERFUL FEATURES. Jinja2 is commonly used with web frameworks such as Flask, Bottle, Morepath, and optionally Django, as well as by configuration management tools like Ansible. The templating engine strikes a thoughtful balance between allowing arbitrary code in templates and giving developers full control.

```
students = [
    {"name": "Willow", "score": 100},
    {"name": "Cordelia", "score": 85},
    {"name": "Oz", "score": 95},
    {"name": "Buffy", "score": 65},
    {"name": "Xander", "score": 45},
]

env = Environment(loader=FileSystemLoader("templates/"))
template = env.get_template("export.csv")

filename = f"output/py_challenge.csv"
context = {
    "students": students,
    "max_score": MAX_SCORE,
}

with open(filename, mode="w", encoding="utf-8") as output:
    output.write(template.render(context))
```

Figure 2: <https://www.youtube.com/watch?v=OraYXEr0Irg>

References

- Pallets. (n.d.). *Jinja2*. <https://palletsprojects.com/p/jinja/>
- PyPI. (n.d.). *Jinja2*. <https://pypi.org/project/Jinja2/>
- Real Python. (2025). *Primer on Jinja Templating*. <https://realpython.com/primer-on-jinja-templating/>
- Better Stack. (n.d.). *Jinja Templating in Python: A Practical Guide*. <https://betterstack.com/community/guides/scaling-python/jinja-templating/>

5 React / JSX

JSX is a syntax extension to JavaScript used with React to describe what the UI should look like, combining the full power of JavaScript with HTML-like markup. Rather than artificially separating technologies by **putting markup and logic in separate files, React uses loosely coupled units called components that contain both rendering logic and markup.** After compilation, JSX expressions become regular JavaScript function calls that evaluate to JavaScript objects, allowing JSX to be used inside if statements and loops, assigned to variables, and returned from functions. JSX looks like HTML but is stricter and can display dynamic information by embedding JavaScript expressions inside curly braces. The Babel compiler transforms JSX down

to `React.createElement()` calls, which create objects called React elements that describe what should appear on screen. JSX is a syntax extension while React is a JavaScript library, and they can be used independently, though they're often used together.

```
main.jsx

function Car() {
  return (
    <>
      <h2>My Car</h2>
      <p>It is a Ford Mustang.</p>
    </>
  );
}

createRoot(document.getElementById('root')).render(
  <Car />
)
```

Run Example »

Inside the components, you can do some other operations before returning the HTML:

Example

JSX in Components:

```
main.jsx

function Car() {
  const brand = "Ford";
  const model = "Mustang";
  return (
    <>
      <h2>My Car</h2>
      <p>It is a {brand} {model}.</p>
    </>
  );
}

createRoot(document.getElementById('root')).render(
  <Car />
)
```

Figure 3: https://www.w3schools.com/react/react_jsx.asp

References

- React Team. (n.d.). *Introducing JSX – React*. <https://legacy.reactjs.org/docs/introducing-jsx.html>
- React Team. (n.d.). *Writing Markup with JSX – React*. <https://react.dev/learn/writing-markup-with-jsx>
- Dodds, K. C. (n.d.). *React/JSX as a server-side templating language*. <https://kentcdodds.com/blog/react-jsx-as-a-server-side-templating-language>

6 Svelte

Svelte is a compiler-based framework that shifts the work of building web apps from the user’s browser to a compile step, producing highly optimized vanilla JavaScript with smaller application bundles and better performance. Unlike frameworks like React and Vue that do work in the browser while apps run, Svelte generates optimal JavaScript code without any runtime overhead. Svelte extends HTML by allowing JavaScript expressions in markup and providing directives for conditions and loops in a fashion similar to Handlebars, extends CSS with a scoping mechanism, and extends JavaScript to achieve true reactivity. The framework provides a developer experience that is more approachable for people with limited experience of the modern tooling ecosystem while sticking closely to the classic web development model of HTML, CSS, and JavaScript. Svelte’s main current disadvantages are that it’s a young framework with a more limited ecosystem compared to mature frameworks, but its performance advantages and shallow learning curve make it attractive for many use cases.

References

- Svelte Team. (n.d.). *Docs • Svelte*. <https://svelte.dev/docs>
- Mozilla. (n.d.). *Getting started with Svelte - Learn web development | MDN*. https://developer.mozilla.org/en-US/docs/Learn_web_development/Core/Frameworks_libraries/Svelte_getting_started
- Creative Tim. (2021). *10+ Svelte Examples - Components & Templates*. <https://www.creative-tim.com/blog/web-development/svelte-examples-components-templates/>

7 Summary Comparison

These alternatives to Mustache represent different philosophies in templating:

- **Nunjucks** and **Handlebars** offer incremental improvements over Mustache with richer syntax while maintaining declarative approaches
- **EJS** provides the full power of JavaScript embedded directly in templates
- **Jinja2** and **Liquid** focus on security and sandboxing for user-editable templates
- **React/JSX** integrates templates directly into JavaScript as first-class code
- **Svelte** compiles templates at build time for optimal runtime performance

The choice depends on your specific needs: logic complexity requirements, security concerns, language ecosystem (JavaScript vs. Python vs. Ruby), performance priorities, and whether you prefer compile-time or runtime template processing.